



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





09

Wprowadzenie do testowania

10 bardzo kosztownych błędów



01 Błąd w systemie Therac-25 (1985-1987)

W maszynach do radioterapii, programowe błędy powodowały nadmierne dawki promieniowania, co doprowadziło do co najmniej 5 śmierci i poważnych obrażeń. Głównym problemem był brak odpowiednich zabezpieczeń w oprogramowaniu.

Odpowiedzialni: Atomic Energy of Canada Limited (AECL) i US Food and Drug Administration (USA)

10 bardzo kosztownych błędów



02 Awaria sieci AT&T (1990)

Błąd w aktualizacji oprogramowania i w NY 114 switchów pada i wstaje co 6 sekund, ~60 000 ludzi nie ma dostępu do połączeń zamiejscowych przez 9h

Odpowiedzialni: AT&T (USA)

10 bardzo kosztownych błędów



03 Błąd w systemie Patriot (1991)

Podczas I wojny w Zatoce Perskiej awaria oprogramowania systemu obronnego spowodowała niedokładne obliczenie trajektorii rakiety i pocisk Scud spadł na koszarzy Amerykanów, a Patriot nie zareagował (zabitych 28 żołnierzy).

Odpowiedzialni: Departament Obrony USA
i Raytheon (USA)

10 bardzo kosztownych błędów



04 Awaria Ariane 5 (1996)

Rakieta eksplodowała 37 sekund po starcie z powodu błędu w konwersji liczby zmiennoprzecinkowej. Straty wyniosły około 370 milionów dolarów.

Odpowiedzialni: Europejska Agencja Kosmiczna), Arianespace (Francja)

10 bardzo kosztownych błędów



05 Awaria Mars Climate Orbiter (1999)

Rozbieżności między systemami jednostek metrycznych w programie przetwarzającym instrukcje kontroli naziemnej (funty) i oprogramowaniem sondy (niutony) spowodowały utratę sondy wartej 125 milionów dolarów.

Odpowiedzialni: NASA i Lockheed Martin (USA)

10 bardzo kosztownych błędów



06 Błąd Millennium (Y2K, 2000)

Skutki były mniej katastrofalne niż przewidywano ...
kosztując miliardy dolarów.

Odpowiedzialni: reprezentacja daty w systemach
informatycznych

10 bardzo kosztownych błędów



07 Hughes Satellite Galaxy IV (2000)

Błąd w oprogramowaniu satelity komunikacyjnego Galaxy IV spowodował zakłócenie sygnału pagerów w całych USA, dotykając 45 milionów użytkowników.

Odpowiedzialni: Hughes Electronics (USA)

10 bardzo kosztownych błędów



08 Virus Slammer (2003)

Wirus komputerowy zaatakował systemy baz danych SQL, powodując globalne zakłócenia w systemach bankowych, lotniczych i wojskowych, generując miliardy dolarów strat.

Odpowiedzialni: błędy w błędach w Microsoft SQL Server

10 bardzo kosztownych błędów



09 Błąd w systemie Knight Capital (2012)

Błąd w algorytmach handlowych doprowadził do strat w wysokości 440 milionów dolarów w ciągu 45 minut, co niemal doprowadziło firmę do bankructwa.

Odpowiedzialni: Knight Capital Group (USA)

10 bardzo kosztownych błędów



10 Awaria Boeing 737 MAX (2018-2020)

Problemy z systemem MCAS, odpowiedzialnym za stabilność samolotu, doprowadziły do dwóch katastrof lotniczych (Lion Air Flight 610 i Ethiopian Airlines Flight 302). System błędnie interpretował dane z jednego czujnika kąta natarcia, co skutkowało automatycznym skierowaniem samolotu w dół. Życie straciło **346** osób.

Odpowiedzialni: Boeing(USA)

Definicje

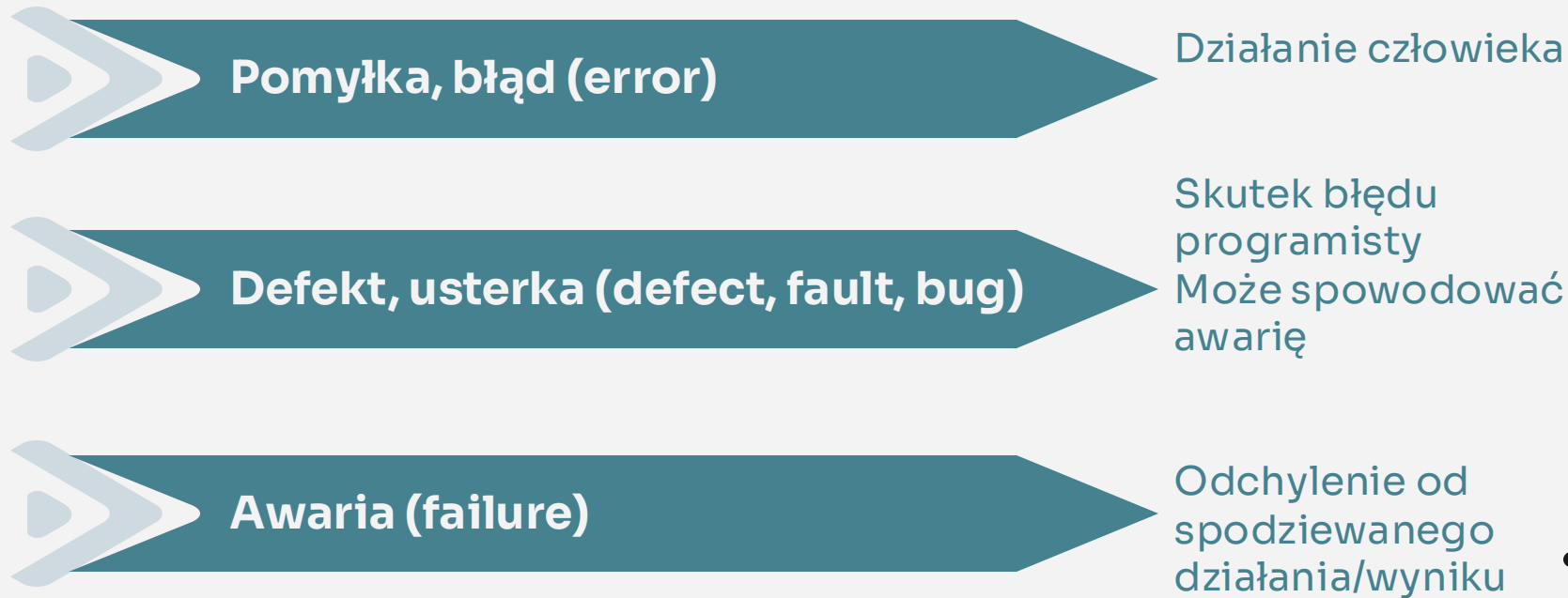
Pomyłka, błąd (error)

Defekt, usterka (defect, fault, bug)

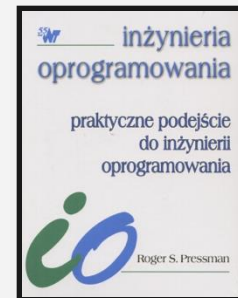
Awaria (failure)



Definicje



Definicje



WERYKIFACJA

- ✓ Czy budujemy produkt tak, jak trzeba?
- ✓ Czy oprogramowanie poprawnie realizuje wyspecyfikowaną funkcję?
- ✓ Zgodność z dokumentacją

WALIDACJA

- ✓ Czy budujemy ten produkt, co trzeba?
- ✓ Porównanie działania gotowego oprogramowania z wymaganiami klientów
- ✓ Zgodność z oczekiwaniami klienta

Definicje

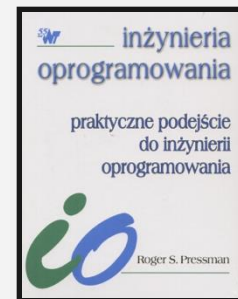


WERYKIFACJA



WALIDACJA

- ✓ Powinny być wykonywane w każdym kroku procesu tworzenia oprogramowania
- ✓ Zaczynają się od przeglądów wymagań, poprzez przeglądy projektów i kontrolę kodu, a kończą się testowaniem produktu



Analiza kodu

01 Statyczna – inspekcje kodu

Praca z kodem źródłowym

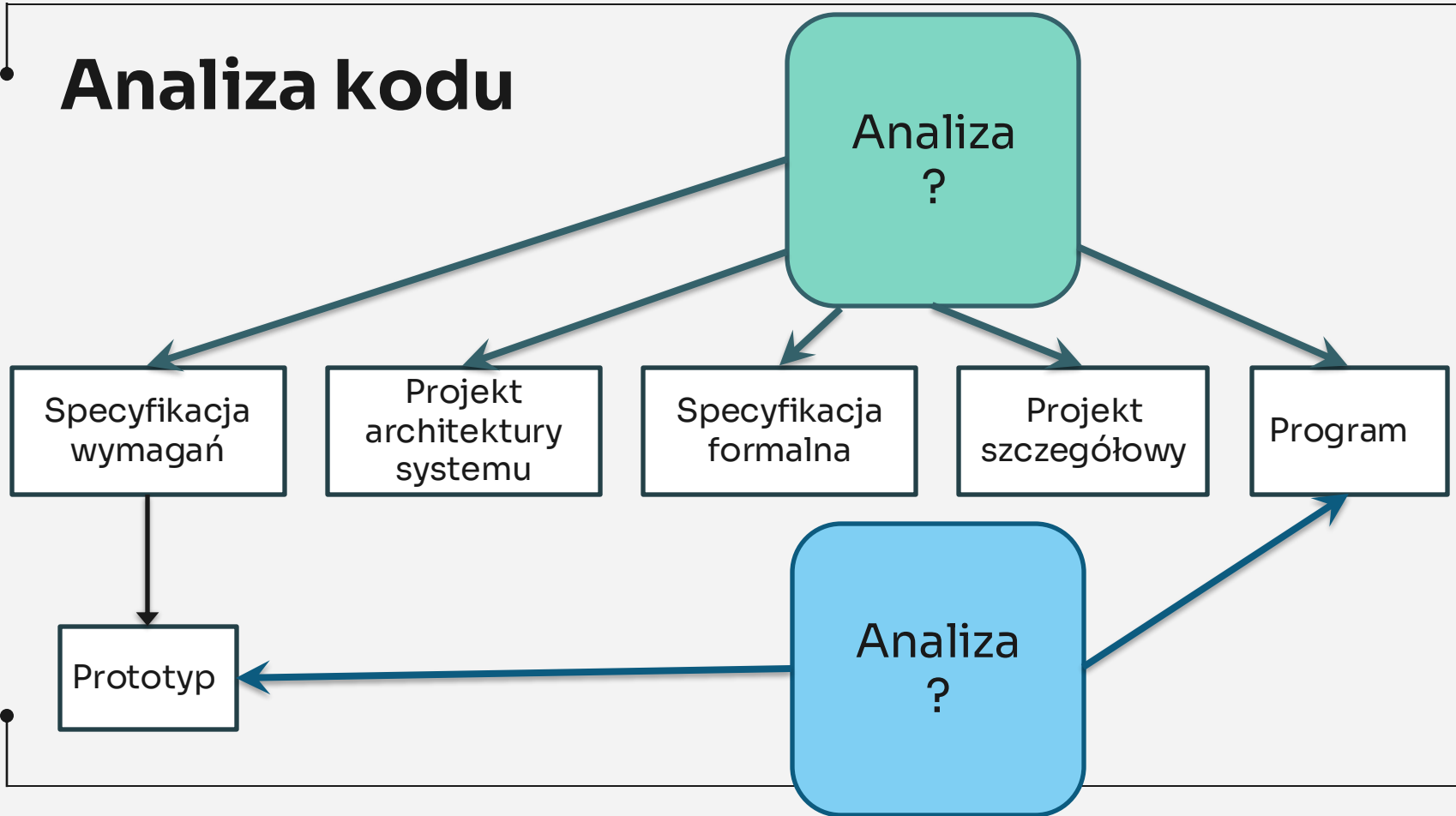


02 Dynamiczna – testowanie

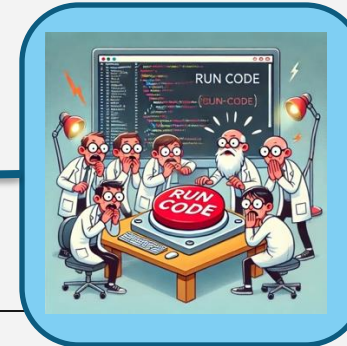
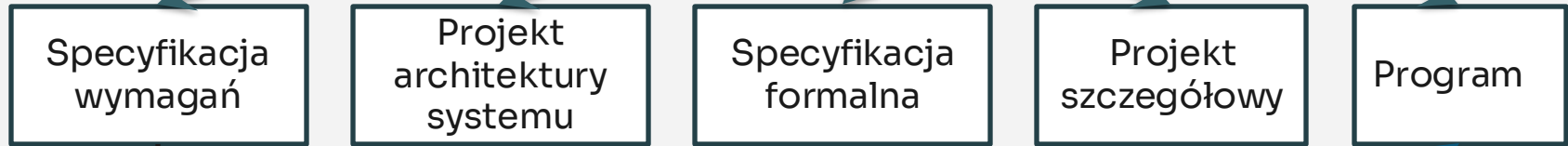
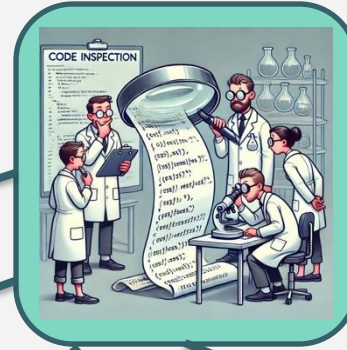
Eksperymentowanie z działającym kodem programu



Analiza kodu



Analiza kodu



Analiza kodu



Całkowita (pełna) weryfikacja i walidacja

Analiza statyczna + dynamiczna



Testowanie konieczne jest do
sprawdzenie jakiej grupy wymagań?



Analiza kodu



Całkowita (pełna) weryfikacja i walidacja

Analiza statyczna + dynamiczna

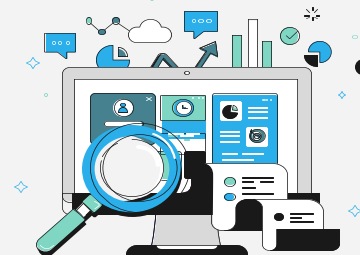


Testowanie konieczne jest do
sprawdzenie jakiej grupy wymagań?

NIEFUNKCJONALNE



Cele testowania



Testowanie polega na:	
Dobry test to taki, który:	
Test jest skuteczny:	

Cele testowania



Testowanie polega na:

uruchamianiu programu w celu wykrycia błędów

Dobry test to taki, który:

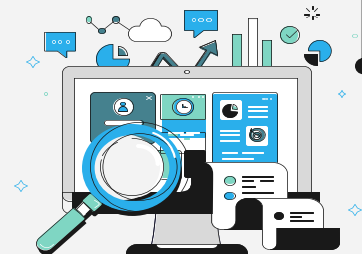
Test jest skuteczny:

Cele testowania



Testowanie polega na:	uruchamianiu programu w celu wykrycia błędów
Dobry test to taki, który:	z dużym prawdopodobieństwem pozwala znaleźć błąd wcześniej nie wykryty
Test jest skuteczny:	

Cele testowania



Testowanie polega na:	uruchamianiu programu w celu wykrycia błędów
Dobry test to taki, który:	z dużym prawdopodobieństwem pozwala znaleźć błąd wcześniej nie wykryty
Test jest skuteczny:	doprowadził do wykrycia nowych błędów

Zasady testowania

01 Buduj testy na podstawie wymagań klienta

02 Planuj!

03 Zasada Pareto (80/20)

04 Testuj bottom-up

05 Gruntowne testowanie jest niewykonalne

06 Powierz testowanie niezależnym osobom



!Gruntownie >> Poziom zaufania

⚙️ **Funkcja oprogramowania:** jak bardzo krytyczny jest system dla firmy

⚙️ **Oczekiwania użytkowników:** jak wiele potrafi znieść użytkownik 😊

⚙️ **Rynek:** lepiej wypuścić szybciej z błędami



Dobry test

- Daje duże prawdopodobieństwo wykrycia błędu
- Ani zbyt prosty, ani zbyt skomplikowany
- Testy nie dublują się
- Wybieramy testy „najlepsze w swoim rodzaju”



Poziomy testowania



Testowanie jednostkowe	Pojedynczych funkcji lub innych fragmentów kodu w oderwaniu od reszty systemu
Testowanie integracyjne	Przetestowane jednostki kodu są stopniowo łączone w większą całość, a następnie ponownie testowane już jako grupa jednostek, proces łączenia i testowania jest powtarzany aż do powstania całego systemu
Testowanie systemowe	Czy system jako całość spełnia wymagania funkcjonalne i jakościowe postawione przez klienta
Testy akceptacyjne	Testy w środowisku docelowym

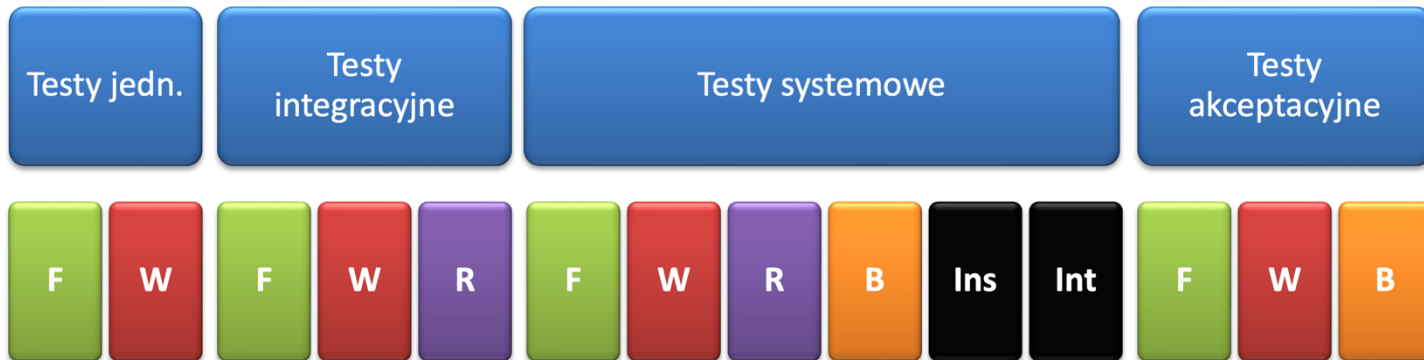
• Typy testów



Funkcjonalne (F)	Co robi?
Niefunkcjonalne (NF)	Jak działa?
Strukturalne (S)	Jaka jest architektura?
Regresywne (R)	Czy nadal działa?



Poziomy a typy testów



F – funkcjonalne
W – wydajnościowe
R – regresywne
B – bezpieczeństwa
Ins – instalacyjne
Int - interfejsu

Projektowanie testów



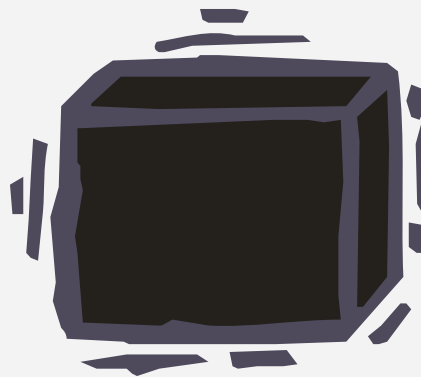
Metoda białej skrzynki

Logiczna struktura algorytmów

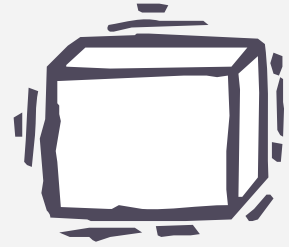


Metoda czarnej skrzynki

Badanie interfejsów i funkcjonalności



Biała skrzynka



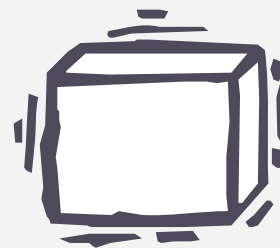
Metoda szklanej skrzynki, testowanie strukturalne



Projektowanie testów na podstawie logicznej struktury algorytmu

- > Wszystkie niezależne ścieżki przebiegu
- > Wszystkie możliwe kombinacje decyzji logicznych
- > Jedno i wielokrotne wykonanie pętli z uwzględnieniem warunków brzegowych

Po co biała skrzynka?



Literówki rozmieszczone są losowo

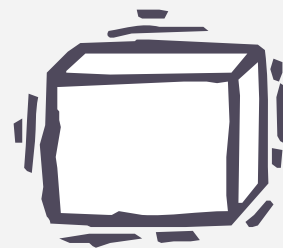


Liczba błędów logicznych i błędnych założeń jest odwrotnie proporcjonalna do częstości wykonywania fragmentu kodu



To, co w założeniu miało być rzadko, w rzeczywistości jest bardzo często wykonywane

Biała skrzynka 100% OK?

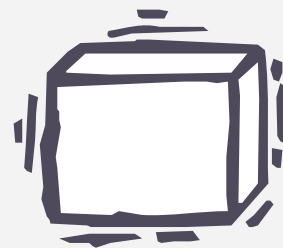


PRZYKŁAD

- > Deklaracja danych
- > 2 zagnieżdżone pętle od 1 do 20 zależnie od wejścia
- > W wewnętrznej pętli program sprawdza 4 warunki

Ile jest możliwych ścieżek?

Biała skrzynka 100% OK?



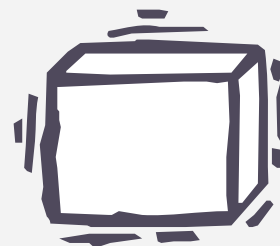
PRZYKŁAD

- > Deklaracja danych
- > 2 zagnieżdżone pętle od 1 do 20 zależnie od wejścia
- > W wewnętrznej pętli program sprawdza 4 warunki

Ile jest możliwych ścieżek?

$\sim 10^{14}$

Biała skrzynka 100% OK?

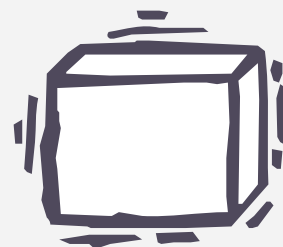


PRZYKŁAD 10^{14}

- > Załóżmy, że istnieje procesor, który może przygotować test, wykonać go i ocenić wynik w ciągu 1 milisekundy
- > Pracując 24h/dobę przez 365 dni w roku

Potrzebowaliby do przetestowania naszego przykładu...

Biała skrzynka 100% OK?



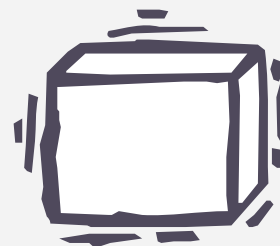
PRZYKŁAD 10^{14}

- > Załóżmy, że istnieje procesor, który może przygotować test, wykonać go i ocenić wynik w ciągu 1 milisekundy
- > Pracując 24h/dobę przez 365 dni w roku

Potrzebowaliby do przetestowania naszego przykładu...

3170 lat

Pokrycie kodu testami



> Pokrycie instrukcji (ang. statement coverage)

Każda instrukcja jest sprawdzana

> Pokrycie gałęzi (ang. branch coverage)

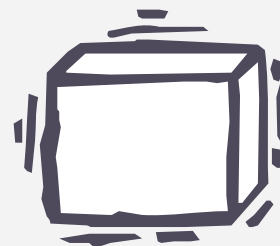
Każda gałąź była odwiedzona

Instrukcja warunkowa musi być przynajmniej raz prawdziwa i przynajmniej raz fałszywa

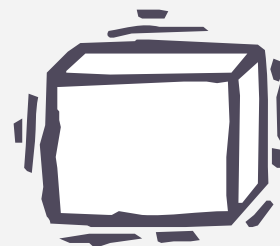


Pokrycie instrukcji

```
void myFunction(int number) {  
    while (liczba<10)  
        liczba++;  
    if (10==liczba)  
        System.out.println(liczba);  
}
```



Pokrycie instrukcji



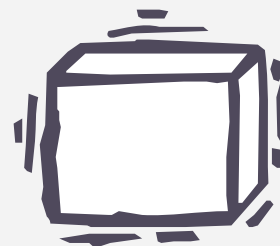
```
void myFunction(int number) {  
    while (liczba<10)  
        liczba++;  
    if (10==liczba)  
        System.out.println(liczba);  
}
```



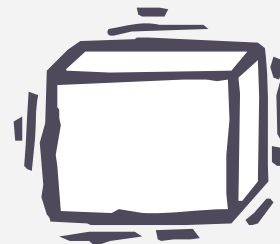
number=3

Pokrycie gałęzi

```
void myFunction(int number) {  
    while (liczba<10)  
        liczba++;  
    if (10==liczba)  
        System.out.println(liczba);  
}
```



Pokrycie gałęzi



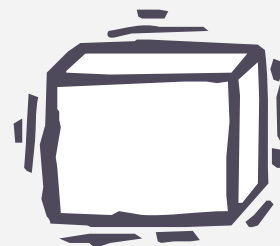
```
void myFunction(int number) {  
    while (liczba<10)  
        liczba++;  
    if (10==liczba)  
        System.out.println(liczba);  
}
```



number=3, number=13

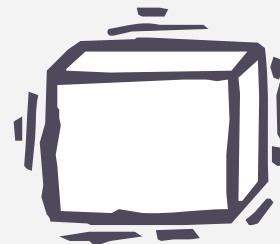
• 100% pokrycia kodu?

```
int add(int no1, int no2) {  
    return no1-no2;  
}
```



• 100% pokrycia kodu?

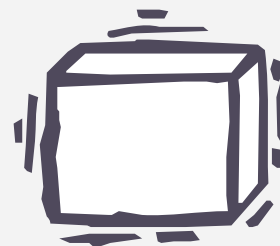
```
int add(int no1, int no2) {  
    return no1-no2;  
}
```



no1=3, no2=0

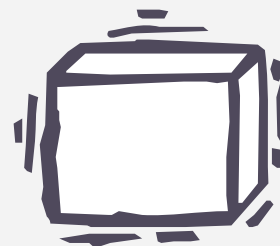
100% pokrycia kodu?

```
int add(int no1, int no2) {  
    return no1-no2;  
    System.out.println(„Martwy kod”);  
}
```



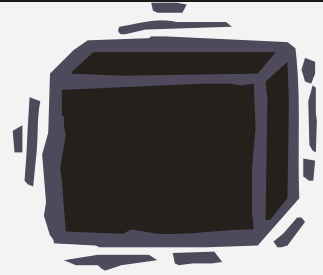
• 100% pokrycia kodu?

```
int add(int no1, int no2) {  
    return no1-no2;  
    System.out.println(„Martwy kod”);  
}
```



• Nie zawsze 100% jest możliwe

Czarna skrzynka



Testowanie zachowania (behawioralne), funkcjonalne

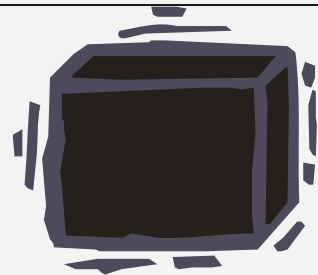


Badanie interfejsów



Nie zwracamy uwagi na wewnętrzną strukturę logiczną

Po co czarna skrzynka?



Wykrycie błędnych interfejsów



Wykrycie pominiętych lub błędnie zaimplementowanych funkcji



Wykrycie błędów w strukturach danych lub metodach dostępu do zewnętrznych baz danych

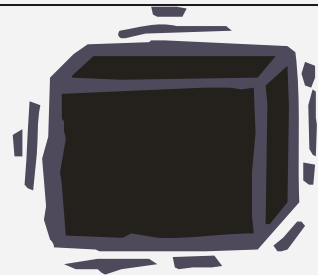


Wykrycie niewłaściwego zachowania lub zbyt małej efektywności systemu



Wykrycie błędów powodujących niewłaściwe uruchamianie się lub wyłączanie systemu

Czarna skrzynka



Stosuje się pod koniec testowania



Pomija się strukturę sterowania



Koncentracja na dziedzinie informacyjnej



Umożliwia przygotowanie testów, które:

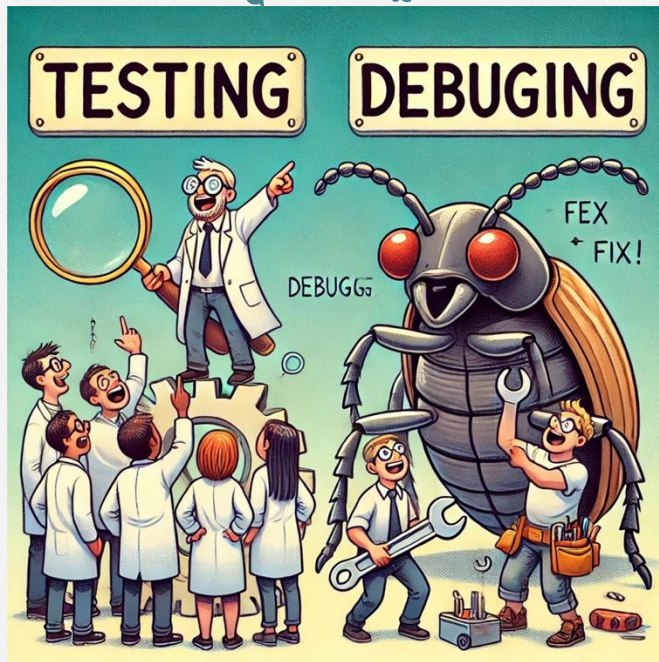
Zmniejszają (>1) liczbę dodatkowych testów koniecznych do przetestowania programu

Mogą wykryć lub wykluczyć całe grupy błędów jednocześnie

• **Testowanie**

VS

Debugowanie

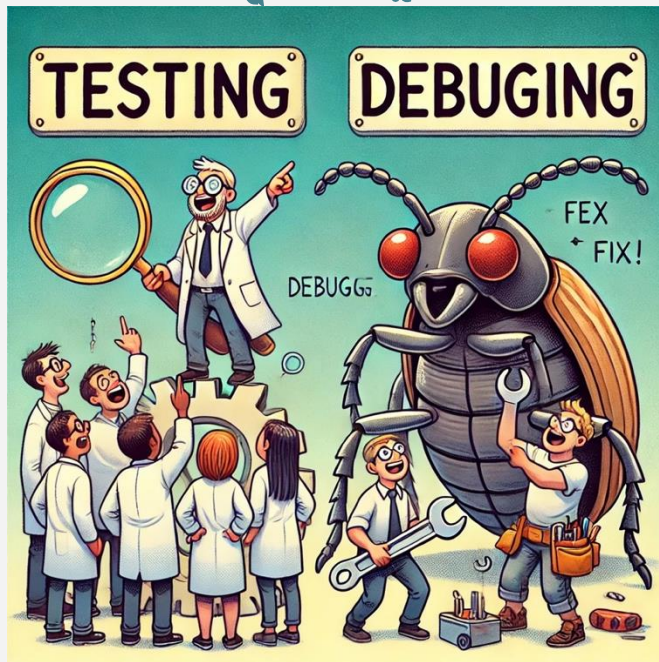


Testowanie

vs

Debugowanie

Ma na celu
wykrycie błędów



Ma na celu
lokalizację i
poprawę błędów

Kiedy koniec testowania?

- ⚙️ Nigdy, jest tylko przekazywane klientowi
- ⚙️ Brakuje na nie czasu lub pieniędzy
- ⚙️ Nie ma idealnej odpowiedzi
- ⚙️ Istnieją praktyczne wskazówki

Kryteria statystyczne: „z 95% pewnością możemy powiedzieć, że prawdopodobieństwo bezbłędnego działania systemu przez 1000h pracy procesora jest równe co najmniej 0,995”





Koniec!